Snappia Medical Coding API

> **Version:** 1.0
> **Base URL:** `https://api.snappia.claims/`
> **Content-Type:** `application/json`

Table of Contents

1. Introduction
2. Authentication
3. Rate Limits
4. Endpoints
   - Submit a Job
   - Get Job Status & Result
   - List Jobs
5. Job Lifecycle
6. Medical Coding Result Schema
   - ICD-10-CM
   - CPT
   - ICD-CPT Linkage
7. Pagination
8. Error Handling
9. Examples
   - Submit and Poll (cURL)
   - Submit and Poll (JavaScript)
   - Submit and Poll (Python)

1. Introduction

The Snappia Medical Coding API processes patient reports and returns structured medical coding results, including **ICD-10-CM** diagnosis codes, **CPT** procedure codes, and **ICD-CPT linkage** mappings.

The API follows an asynchronous, job-based workflow:

1. **Submit** a patient report and receive a job ID.
2. **Poll** the job ID until processing completes.
3. **Retrieve** the structured medical coding results.

All requests and responses use JSON. All timestamps are in ISO 8601 format (UTC).

2. Authentication

All requests must include a Bearer token in the `Authorization` header.

```
Authorization: Bearer <your_api_token>
```

Header	Value
`Authorization`	`Bearer <your_api_token>`
`Content-Type`	`application/json` (required for POST requests)

Authentication errors:

Scenario	Status	Response
Missing `Authorization` header	`403`	`{"detail": "Not authenticated"}`
Invalid or revoked token	`401`	`{"detail": "Invalid or inactive API token"}`

3. Rate Limits

Two rate limits are enforced to ensure service reliability.

Job Submission Limit

Metric	Value
Scope	Per user

Metric	Value
Limit	50 submissions per day
Reset	Midnight UTC

The daily count includes jobs with status `pending`, `processing`, or `completed`. Failed jobs do not count toward your limit.

Your account administrator can adjust this limit.

Polling Limit

Metric	Value
Scope	Per user
Limit	60 requests per minute
Window	Sliding (rolling 60 seconds)
Applies to	`GET /api/v1/jobs/{job_id}` and `GET /api/v1/jobs`

When a rate limit is exceeded, the API returns `429 Too Many Requests`. See Error Handling for response format.

4. Endpoints

Base path: `/api/v1`

Method	Endpoint	Description
`POST`	`/api/v1/jobs`	Submit a patient report for coding
`GET`	`/api/v1/jobs/{job_id}`	Get a single job's status and result
`GET`	`/api/v1/jobs`	List your jobs with filtering and pagination

4.1 Submit a Job

Submit a patient report for medical coding. The request is queued for processing.

```
POST /api/v1/jobs
```

Request body:

Field	Type	Required	Description
`patient_report`	`string`	Yes	The patient report text. Maximum 20,000 characters.

Response: `202 Accepted`

Field	Type	Description
`job_id`	`string` (UUID)	Unique identifier for the submitted job
`status`	`string`	`"pending"`

Example request:

```
curl -X POST https://api.snappia.claims/api/v1/jobs \
  -H "Authorization: Bearer <your_api_token>" \
  -H "Content-Type: application/json" \
  -d '{"patient_report": "Patient is a 55-year-old male presenting with chest pain and shortness
```

Example response:

```
{
  "job_id": "bc6d24cc-bf76-4a70-9099-9cc3f01d9632",
  "status": "pending"
}
```

Errors:

Status	Condition
`401`	Invalid or inactive API token
`422`	Missing or invalid `patient_report` field

Status	Condition
`429`	Daily submission limit reached

4.2 Get Job Status & Result

Retrieve the status and result of a specific job. You can only access jobs you submitted.

```
GET /api/v1/jobs/{job_id}
```

Path parameters:

Parameter	Type	Description
`job_id`	`string` (UUID)	The job identifier returned from submission

Response: `200 OK`

Field	Type	Nullable	Description
`id`	`string` (UUID)	No	Job identifier
`status`	`string`	No	`"pending"`, `"processing"`, `"completed"`, or `"failed"`
`patient_report`	`string`	No	The original input text
`medical_coding_result`	`object`	Yes	Coding results. `null` until status is `"completed"`. See Section 6.
`error_message`	`string`	Yes	Error details. `null` unless status is `"failed"`.
`created_at`	`string` (ISO 8601)	No	When the job was submitted
`updated_at`	`string` (ISO 8601)	No	When the job status last changed

Field	Type	Nullable	Description
`completed_at`	`string` (ISO 8601)	Yes	When processing finished. `null` while pending or processing.

Example request:

```
curl https://api.snappia.claims/api/v1/jobs/bc6d24cc-bf76-4a70-9099-9cc3f01d9632 \
  -H "Authorization: Bearer <your_api_token>"
```

Example response (completed):

```
{
  "id": "bc6d24cc-bf76-4a70-9099-9cc3f01d9632",
  "status": "completed",
  "patient_report": "Patient is a 55-year-old male presenting with chest pain...",
  "medical_coding_result": {
    "icd": {
      "care_setting": "outpatient",
      "rationale": "Based on the clinical documentation...",
      "result": [
        {
          "ICD-10-CM Code": "I20.9",
          "ICD Description": "Angina pectoris, unspecified",
          "Type": "Primary",
          "Order No.": 1,
          "Rationale": "The patient presents with chest pain...",
          "Confidence Score": 95.5
        }
      ]
    },
    "cpt": {
      "rationale": "Based on the level of service documented...",
      "result": [
        {
          "code": "99213",
          "description": "Office or other outpatient visit for evaluation and management...",
          "modifier": null,
          "units": 1,
          "score": 92.0,
          "rationale": "The documentation supports a level 3 E/M visit..."
        }
      ]
    },
    "linkage": {
      "linkage": {
        "99213": {
          "icd": ["I20.9 - Angina pectoris, unspecified"],
          "description": "Office or other outpatient visit...",
          "modifier": null,
          "units": 1,
          "score": 92.0,
          "rationale": "This CPT code is linked to the primary ICD..."
        }
      }
```

```
    }
  },
  "error_message": null,
  "created_at": "2026-02-09T06:08:48.770581+00:00",
  "updated_at": "2026-02-09T06:08:49.006060+00:00",
  "completed_at": "2026-02-09T06:08:49.005060+00:00"
}
```


Example response (pending):


```
{
  "id": "bc6d24cc-bf76-4a70-9099-9cc3f01d9632",
  "status": "pending",
  "patient_report": "Patient is a 55-year-old male presenting with chest pain...",
  "medical_coding_result": null,
  "error_message": null,
  "created_at": "2026-02-09T06:08:48.770581+00:00",
  "updated_at": "2026-02-09T06:08:48.770581+00:00",
  "completed_at": null
}
```


Errors:

Status	Condition
`401`	Invalid or inactive API token
`404`	Job not found or does not belong to your account
`429`	Polling rate limit exceeded (60 requests/minute)

4.3 List Jobs

List your submitted jobs with optional status filtering and pagination.

```
GET /api/v1/jobs
```

Query parameters:

Parameter	Type	Default	Constraints	Description
`status_filter`	`string`	—	`"pending"` , `"processing"` , `"completed"` , or `"failed"`	Filter by job status
`skip`	`integer`	`0`	>= 0	Number of records to skip (offset)
`limit`	`integer`	`5`	1 -- 20	Number of records to return (page size)

Response: `200 OK`

Field	Type	Description
`jobs`	`array`	Array of job objects (same schema as Section 4.2 response)
`total`	`integer`	Total number of matching jobs (for pagination)
`skip`	`integer`	Echo of the `skip` parameter
`limit`	`integer`	Echo of the `limit` parameter

Example request:

```
# All jobs (first page)
curl "https://api.snappia.claims/api/v1/jobs" \
  -H "Authorization: Bearer <your_api_token>"

# Completed jobs only, second page
curl "https://api.snappia.claims/api/v1/jobs?status_filter=completed&skip=5&limit=5" \
  -H "Authorization: Bearer <your_api_token>"
```

Example response:

```
{
  "jobs": [
    {
      "id": "bc6d24cc-bf76-4a70-9099-9cc3f01d9632",
      "status": "completed",
      "patient_report": "Patient presents with...",
      "medical_coding_result": { "icd": {...}, "cpt": {...}, "linkage": {...} },
      "error_message": null,
      "created_at": "2026-02-09T06:08:48.770581+00:00",
      "updated_at": "2026-02-09T06:08:49.006060+00:00",
      "completed_at": "2026-02-09T06:08:49.005060+00:00"
    }
  ],
  "total": 42,
  "skip": 0,
  "limit": 5
}
```


Errors:

Status	Condition
`401`	Invalid or inactive API token
`422`	Invalid query parameter (e.g., `limit` outside 1--20 range)
`429`	Polling rate limit exceeded (60 requests/minute)

5. Job Lifecycle

A job progresses through the following statuses:

```
pending  -->  processing  -->  completed
                           -->  failed
```

Status	Description	`medical_coding_result`	`error_message`
`pending`	Queued and waiting to be processed	`null`	`null`

Status	Description	`medical_coding_result`	`error_message`
`processing`	Actively being processed by the coding engine	`null`	`null`
`completed`	Processing finished successfully	Contains results	`null`
`failed`	Processing encountered an error	`null`	Contains error details

Processing time: Jobs typically take 1--10 minutes depending on report complexity and queue position.

Recommended polling interval: 5 seconds. If you receive a `429` response, back off to 10--15 seconds.

6. Medical Coding Result Schema

When a job reaches `"completed"` status, the `medical_coding_result` field contains three sections: ICD-10-CM diagnosis codes, CPT procedure codes, and ICD-CPT linkage mappings.

```
{
  "icd": { ... },
  "cpt": { ... },
  "linkage": { ... }
}
```

6.1 ICD-10-CM

The `icd` object contains diagnosis coding results.

```json
{
  "icd": {
    "care_setting": "outpatient",
    "result": "Based on the clinical documentation...",
    "result": [
      {
        "ICD-10-CM Code": "I20.9",
        "ICD Description": "Angina pectoris, unspecified",
        "Type": "Primary",
        "Order No.": 1,
        "Rationale": "The patient presents with chest pain...",
        "Confidence Score": 95.5
      }
    ]
  }
}
```

Top-level fields:

Field	Type	Description
`care_setting`	`string`	`"outpatient"` or `"inpatient"`
`rationale`	`string`	Overall explanation of diagnosis code selections
`result`	`array`	List of ICD-10-CM codes

Each item in `result`:

Field	Type	Description
`ICD-10-CM Code`	`string`	The ICD-10-CM code (e.g., `"I20.9"`)
`ICD Description`	`string`	Full description of the diagnosis code
`Type`	`string`	`"Primary"`, `"Additional"`, or `"Secondary"`
`Order No.`	`integer`	Sequencing order (1-based)
`Rationale`	`string`	Clinical justification for this code
`Confidence Score`	`float`	Confidence rating from 0 to 100

6.2 CPT

The `cpt` object contains procedure coding results.

```
{
  "cpt": {
    "rationale": "Based on the level of service documented...",
    "result": [
      {
        "code": "99213",
        "description": "Office or other outpatient visit...",
        "modifier": null,
        "units": 1,
        "score": 92.0,
        "rationale": "The documentation supports a level 3 E/M visit..."
      }
    ]
  }
}
```

Top-level fields:

Field	Type	Description
`rationale`	`string`	Overall rationale for the CPT code selections
`result`	`array`	List of CPT codes

Each item in `result`:

Field	Type	Description
`code`	`string`	CPT code (e.g., `"99213"`)
`description`	`string`	Code description
`modifier`	`string` or `null`	Modifier code (e.g., `"25"`) or `null` if none
`units`	`integer`	Number of units
`score`	`float`	Confidence rating from 0 to 100
`rationale`	`string`	Code-specific justification

6.3 ICD-CPT Linkage

The `linkage` object maps each CPT code to its supporting ICD diagnosis codes.

```
{
  "linkage": {
    "linkage": {
      "99213": {
        "icd": ["I20.9 - Angina pectoris, unspecified"],
        "description": "Office or other outpatient visit...",
        "modifier": null,
        "units": 1,
        "score": 92.0,
        "rationale": "This CPT code is linked to the primary ICD..."
      }
    }
  }
}
```

The outer `linkage` object contains a nested `linkage` object whose keys are CPT codes.

Each CPT code entry:

Field	Type	Description
`icd`	`array` of `string`	Linked ICD diagnosis entries
`description`	`string`	CPT code description
`modifier`	`string` or `null`	Modifier code or `null`
`units`	`integer`	Number of units
`score`	`float`	Confidence rating from 0 to 100
`rationale`	`string`	Justification for the linkage

7. Pagination

List endpoints support offset-based pagination.

Parameter	Default	Range	Description
`skip`	`0`	>= 0	Records to skip
`limit`	`5`	1 -- 20	Records per page

Calculating total pages:

```
total_pages = ceil(total / limit)
current_page = floor(skip / limit) + 1
```

Iterating through all pages:

```
Page 1:  ?skip=0&limit=5
Page 2:  ?skip=5&limit=5
Page 3:  ?skip=10&limit=5
...
```

8. Error Handling

All error responses follow a consistent JSON format.

Standard Error

```
{
  "detail": "Human-readable error message"
}
```

Validation Error (422)

Returned when the request body or query parameters fail validation.

```
{
  "detail": [
    {
      "type": "missing",
      "loc": ["body", "patient_report"],
      "msg": "Field required",
      "input": {}
    }
  ]
}
```

Field	Type	Description
`detail[].type`	`string`	Error type (e.g., `"missing"` , `"string_type"`)
`detail[].loc`	`array`	Path to the invalid field
`detail[].msg`	`string`	Human-readable error message
`detail[].input`	`any`	The value that was provided

HTTP Status Code Reference

Code	Meaning	When
`200`	OK	Successful retrieval
`202`	Accepted	Job submitted and queued
`401`	Unauthorized	Invalid or inactive API token
`403`	Forbidden	Missing `Authorization` header
`404`	Not Found	Job does not exist or belongs to another account
`422`	Validation Error	Invalid request body or query parameters
`429`	Too Many Requests	Rate limit exceeded

9. Examples

9.1 Submit and Poll (cURL)

```
# 1. Submit a job
JOB_RESPONSE=$(curl -s -X POST https://api.snappia.claims/api/v1/jobs \
  -H "Authorization: Bearer <your_api_token>" \
  -H "Content-Type: application/json" \
  -d '{"patient_report": "Patient is a 55-year-old male presenting with chest pain and shortness

JOB_ID=$(echo $JOB_RESPONSE | jq -r '.job_id')
echo "Job submitted: $JOB_ID"

# 2. Poll until complete
while true; do
  RESULT=$(curl -s https://api.snappia.claims/api/v1/jobs/$JOB_ID \
    -H "Authorization: Bearer <your_api_token>")

  STATUS=$(echo $RESULT | jq -r '.status')
  echo "Status: $STATUS"

  if [ "$STATUS" = "completed" ] || [ "$STATUS" = "failed" ]; then
    echo "$RESULT" | jq .
    break
  fi

  sleep 5
done
```

9.2 Submit and Poll (JavaScript)

```javascript
const API_BASE = "https://api.snappia.claims/api/v1";
const TOKEN = "<your_api_token>";

const headers = {
  "Authorization": `Bearer ${TOKEN}`,
  "Content-Type": "application/json",
};

// Submit a job
async function submitJob(patientReport) {
  const response = await fetch(`${API_BASE}/jobs`, {
    method: "POST",
    headers,
    body: JSON.stringify({ patient_report: patientReport }),
  });

  if (response.status === 429) {
    throw new Error("Daily submission limit reached");
  }
  if (!response.ok) {
    throw new Error(`Submission failed: ${response.status}`);
  }

  return response.json(); // { job_id, status }
}

// Poll for results
async function pollJob(jobId, intervalMs = 5000) {
  while (true) {
    const response = await fetch(`${API_BASE}/jobs/${jobId}`, { headers });

    if (response.status === 429) {
      await new Promise((r) => setTimeout(r, 15000));
      continue;
    }
    if (!response.ok) {
      throw new Error(`Poll failed: ${response.status}`);
    }

    const job = await response.json();
```

```
    if (job.status === "completed" || job.status === "failed") {
      return job;
    }

    await new Promise((r) => setTimeout(r, intervalMs));
  }
}

// Usage
const { job_id } = await submitJob("Patient presents with acute bronchitis...");
const result = await pollJob(job_id);

if (result.status === "completed") {
  console.log("ICD codes:", result.medical_coding_result.icd.result);
  console.log("CPT codes:", result.medical_coding_result.cpt.result);
} else {
  console.error("Job failed:", result.error_message);
}
```

9.3 Submit and Poll (Python)

```python
import time
import requests

API_BASE = "https://api.snappia.claims/api/v1"
TOKEN = "<your_api_token>"

headers = {
    "Authorization": f"Bearer {TOKEN}",
    "Content-Type": "application/json",
}


def submit_job(patient_report: str) -> str:
    """Submit a patient report and return the job ID."""
    response = requests.post(
        f"{API_BASE}/jobs",
        headers=headers,
        json={"patient_report": patient_report},
    )
    response.raise_for_status()
    return response.json()["job_id"]


def poll_job(job_id: str, interval: int = 5) -> dict:
    """Poll until the job reaches a terminal status."""
    while True:
        response = requests.get(f"{API_BASE}/jobs/{job_id}", headers=headers)

        if response.status_code == 429:
            time.sleep(15)
            continue

        response.raise_for_status()
        job = response.json()

        if job["status"] in ("completed", "failed"):
            return job

        time.sleep(interval)
```

```python
# Usage
job_id = submit_job("Patient is a 55-year-old male presenting with chest pain...")
result = poll_job(job_id)

if result["status"] == "completed":
    icd_codes = result["medical_coding_result"]["icd"]["result"]
    cpt_codes = result["medical_coding_result"]["cpt"]["result"]

    for code in icd_codes:
        print(f"{code['ICD-10-CM Code']}: {code['ICD Description']} "
              f"(Confidence: {code['Confidence Score']}%)")

    for code in cpt_codes:
        print(f"{code['code']}: {code['description']} "
              f"(Confidence: {code['score']}%)")
else:
    print(f"Job failed: {result['error_message']}")
```